

DSA0404

1. **Scenario:** You are working on a project that involves analyzing a dataset containing information about houses in a neighborhood. The dataset is stored in a CSV file, and you have imported it into a NumPy array named `house_data`. Each row of the array represents a house, and the columns contain various features such as the number of bedrooms, square footage, and sale price. **Question:** Using NumPy arrays and operations, how would you find the averagesale price of houses with more than four bedrooms in the neighborhood?

Python code :

```
import numpy as np
```

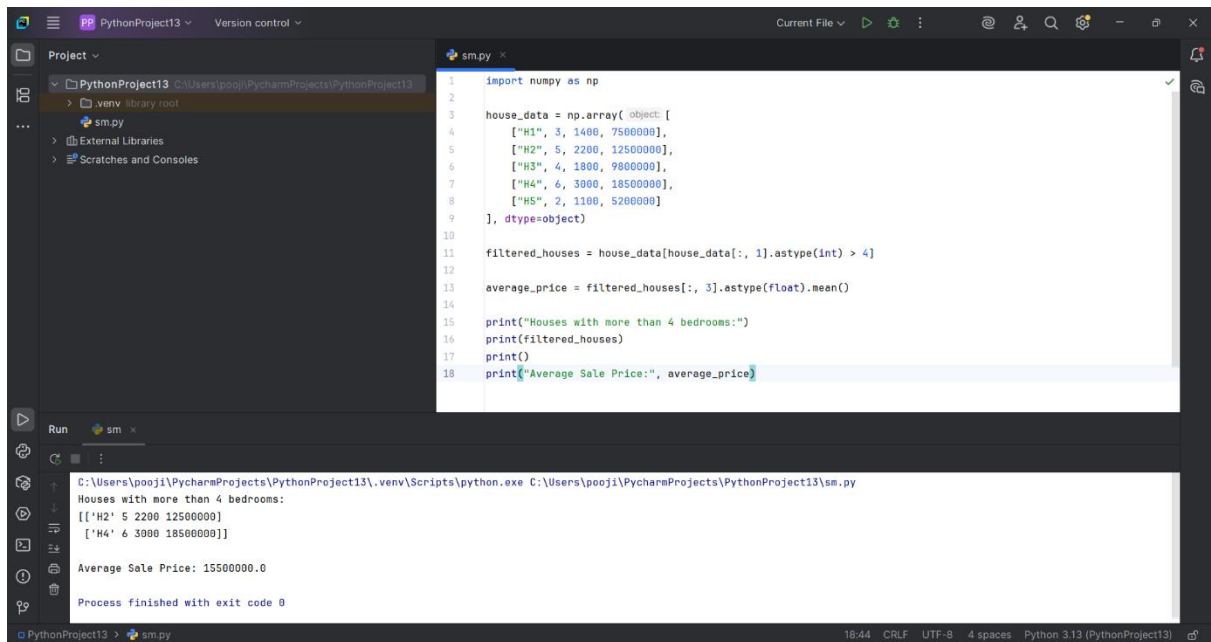
```
house_data = np.array([
    ["H1", 3, 1400, 7500000],
    ["H2", 5, 2200, 12500000],
    ["H3", 4, 1800, 9800000],
    ["H4", 6, 3000, 18500000],
    ["H5", 2, 1100, 5200000]
], dtype=object)
```

```
filtered_houses = house_data[house_data[:, 1].astype(int) > 4]
```

```
average_price = filtered_houses[:, 3].astype(float).mean()
```

```
print("Houses with more than 4 bedrooms:")
print(filtered_houses)
print()
print("Average Sale Price:", average_price)
```

OUTPUT :



The screenshot shows a PyCharm IDE window for a project named 'PythonProject13'. The editor displays a file named 'sm.py' with the following Python code:

```
1 import numpy as np
2
3 house_data = np.array( object: [
4     ["H1", 3, 1400, 7500000],
5     ["H2", 5, 2200, 12500000],
6     ["H3", 4, 1800, 9000000],
7     ["H4", 6, 3000, 18500000],
8     ["H5", 2, 1100, 5200000]
9 ], dtype=object)
10
11 filtered_houses = house_data[house_data[:, 1].astype(int) > 4]
12
13 average_price = filtered_houses[:, 3].astype(float).mean()
14
15 print("Houses with more than 4 bedrooms:")
16 print(filtered_houses)
17 print()
18 print("Average Sale Price:", average_price)
```

The Run window at the bottom shows the output of the script:

```
C:\Users\pooji\PycharmProjects\PythonProject13\.venv\Scripts\python.exe C:\Users\pooji\PycharmProjects\PythonProject13\sm.py
Houses with more than 4 bedrooms:
[["H2" 5 2200 12500000]
 ["H4" 6 3000 18500000]]

Average Sale Price: 15500000.0

Process finished with exit code 0
```

2. **Scenario: An email service provider wants to automatically classify incoming emails as spam or non-spam. Each email is represented using features such as number of links, number of capitalized words, and total email length. Historical email data is labeled as spam (1) or not spam (0). Question: Develop a Logistic Regression model using the given features. Predict whether a newly received email is spam or not.**

Python code :

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
```

```
data = {
    "Num_Links": [1, 6, 0, 4, 2],
    "Num_Caps_Words": [3, 15, 1, 10, 5],
    "Email_Length": [120, 300, 80, 250, 160],
    "Spam": [0, 1, 0, 1, 0]
}
```

```
df = pd.DataFrame(data)
```

```
X = df[["Num_Links", "Num_Caps_Words", "Email_Length"]]
```

```
y = df["Spam"]
```

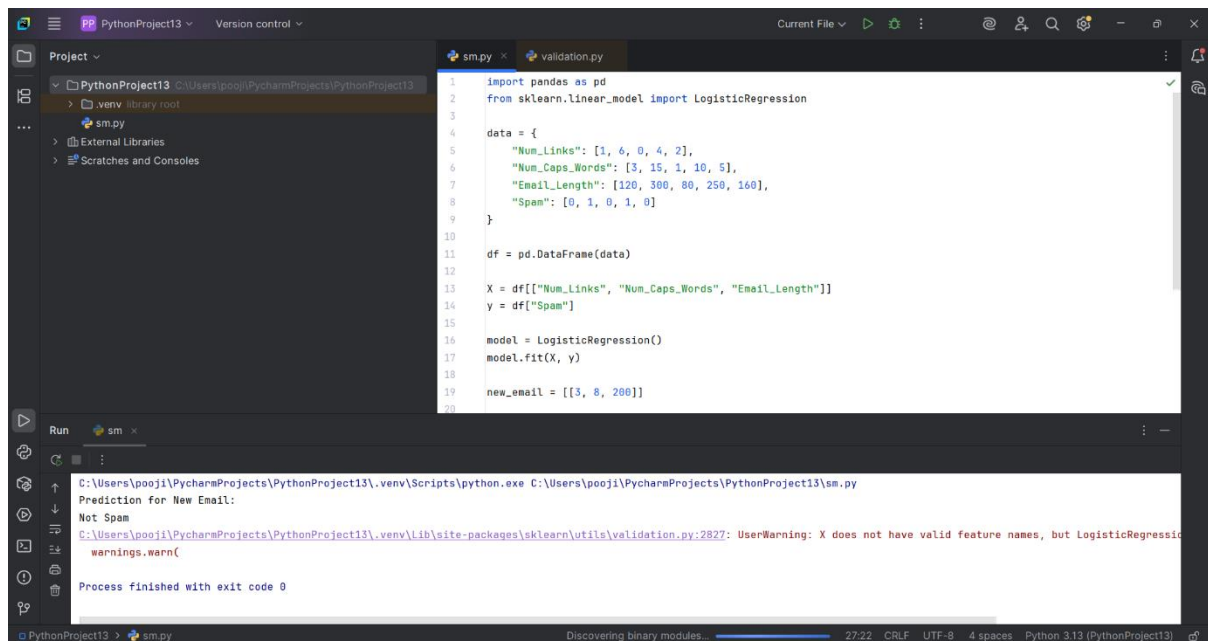
```
model = LogisticRegression()  
model.fit(X, y)
```

```
new_email = [[3, 8, 200]]
```

```
prediction = model.predict(new_email)
```

```
print("Prediction for New Email:")  
if prediction[0] == 1:  
    print("Spam")  
else:  
    print("Not Spam")
```

OUTPUT :



The screenshot shows a PyCharm IDE window for a project named 'PythonProject13'. The editor displays a file named 'sm.py' with the following code:

```
1 import pandas as pd  
2 from sklearn.linear_model import LogisticRegression  
3  
4 data = {  
5     "Num_Links": [1, 6, 0, 4, 2],  
6     "Num_Caps_Words": [3, 15, 1, 10, 5],  
7     "Email_Length": [120, 300, 80, 250, 160],  
8     "Spam": [0, 1, 0, 1, 0]  
9 }  
10  
11 df = pd.DataFrame(data)  
12 X = df[["Num_Links", "Num_Caps_Words", "Email_Length"]]  
13 y = df["Spam"]  
14  
15  
16 model = LogisticRegression()  
17 model.fit(X, y)  
18  
19 new_email = [[3, 8, 200]]  
20
```

The Run console at the bottom shows the execution output:

```
C:\Users\pooji\PycharmProjects\PythonProject13\.venv\Scripts\python.exe C:\Users\pooji\PycharmProjects\PythonProject13\sm.py  
Prediction for New Email:  
Not Spam  
C:\Users\pooji\PycharmProjects\PythonProject13\.venv\Lib\site-packages\sklearn\utils\validation.py:2827: UserWarning: X does not have valid feature names, but LogisticRegression  
warnings.warn(  
Process finished with exit code 0
```

The status bar at the bottom indicates the file encoding is UTF-8, the line length is 27/22, and the Python version is 3.13.

3. **Scenario:** An email service provider wants to automatically classify incoming emails as spam or non-spam. Each email is represented using features such as number of links, number of capitalized words, and total email length. Historical email data is labeled as spam (1) or not spam (0). **Question:** Develop a Logistic Regression model using the given features. Predict whether a newly received email is spam or not.

Python code :

```
import pandas as pd
from sklearn.linear_model import LogisticRegression

data = {
    "Num_Links": [1, 6, 0, 4, 2],
    "Num_Caps_Words": [3, 15, 1, 10, 5],
    "Email_Length": [120, 300, 80, 250, 160],
    "Spam": [0, 1, 0, 1, 0]
}

df = pd.DataFrame(data)

X = df[["Num_Links", "Num_Caps_Words", "Email_Length"]]
y = df["Spam"]

model = LogisticRegression()
model.fit(X, y)

new_email = [[3, 8, 200]]

prediction = model.predict(new_email)

print("Prediction for New Email:")
if prediction[0] == 1:
    print("Spam")
else:
    print("Not Spam")
```

OUTPUT :

```
1 import pandas as pd
2 from sklearn.linear_model import LogisticRegression
3
4 data = {
5     "Num_Links": [1, 6, 0, 4, 2],
6     "Num_Caps_Words": [3, 15, 1, 10, 5],
7     "Email_Length": [120, 300, 80, 250, 160],
8     "Spam": [0, 1, 0, 1, 0]
9 }
10
11 df = pd.DataFrame(data)
12
13 X = df[["Num_Links", "Num_Caps_Words", "Email_Length"]]
14 y = df["Spam"]
15
16 model = LogisticRegression()
17 model.fit(X, y)
18
19 new_email = [[3, 8, 200]]
```

Run

```
C:\Users\pooji\PycharmProjects\PythonProject13\.venv\Scripts\python.exe C:\Users\pooji\PycharmProjects\PythonProject13\sm.py
Prediction for New Email:
Not Spam
C:\Users\pooji\PycharmProjects\PythonProject13\.venv\lib\site-packages\sklearn\utils\validation.py:2827: UserWarning: X does not have valid feature names, but LogisticRegression
warnings.warn(
Process finished with exit code 0
```

4. **Scenario: Linear Regression for Car Resale Price Prediction** You are a data analyst at a used-car marketplace. You want to predict the resale price of a car based on features such as car age (years), total kilometers driven, and engine capacity (cc). You have a dataset containing past car listings with these features and their final selling prices. **Question:** Write a Python program that loads the dataset containing Age,Kms_Driven, Engine_CC, and Resale_Price, trains a Linear Regression model using scikit-learn, accepts user input for a new car, and predicts the estimated resale price.

Python code :

```
import numpy as np
from sklearn.linear_model import LinearRegression
car_models = np.array([
    "Swift",
    "City",
    "i20",
```

```
"Creta",  
"Alto"  
])
```

```
X = np.array([  
    [2, 25000, 1200],  
    [5, 60000, 1500],  
    [3, 40000, 1300],  
    [7, 90000, 1800],  
    [1, 15000, 1000]  
])
```

```
y = np.array([650000, 420000, 520000, 300000, 720000])
```

```
model = LinearRegression()
```

```
model.fit(X, y)
```

```
print("Available Car Models:")
```

```
for i in range(len(car_models)):
```

```
    print(i + 1, "-", car_models[i])
```

```
car_model = input("Enter Car Model: ")
```

```
age = int(input("Enter Car Age (Years): "))
```

```
kms = int(input("Enter Kilometers Driven: "))
```

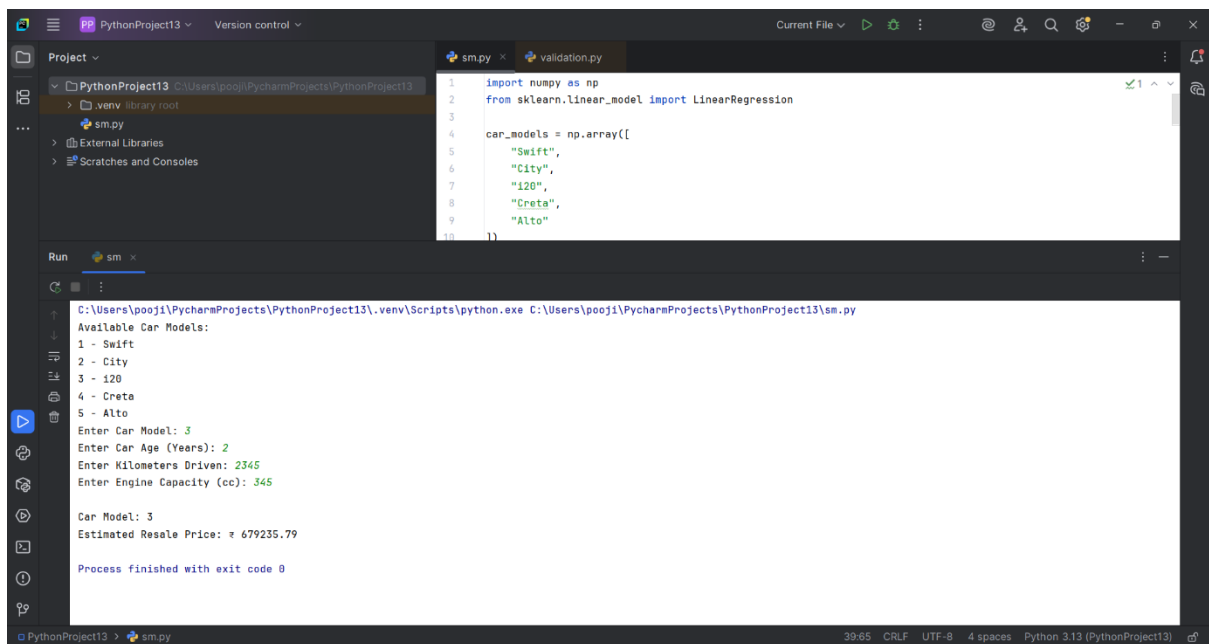
```
engine = int(input("Enter Engine Capacity (cc): "))
```

```
new_car = np.array([[age, kms, engine]])
```

```
predicted_price = model.predict(new_car)
```

```
print("\nCar Model:", car_model)
```

```
print("Estimated Resale Price: ₹", round(predicted_price[0], 2))
```



5. Scenario: Loan Approval (CART Classification): A bank wants to decide whether to approve or reject a loan application. The decision depends on features like income, credit score, and existing loans. Past applications are labeled as Approved (1) or Rejected (0).

Question: Using CART, build a Decision Tree Classifier for loan approval. Predict the class for a new applicant and display the decision rules.

Python code :

```
import numpy as np
```

```
from sklearn.tree import DecisionTreeClassifier, export_text
```

```

X = np.array([
    [45000, 720, 0],
    [30000, 650, 1],
    [60000, 780, 0],
    [28000, 620, 2],
    [52000, 710, 1]
])

```

```

y = np.array([1, 0, 1, 0, 1])

model = DecisionTreeClassifier(criterion="gini", random_state=42)

model.fit(X, y)

income = int(input("Enter Monthly Income (₹): "))

credit_score = int(input("Enter Credit Score (0-900): "))

existing_loans = int(input("Enter Number of Existing Loans: "))

new_applicant = np.array([[income, credit_score, existing_loans]])

prediction = model.predict(new_applicant)

if prediction[0] == 1:

    print("\nLoan Approved")

else:

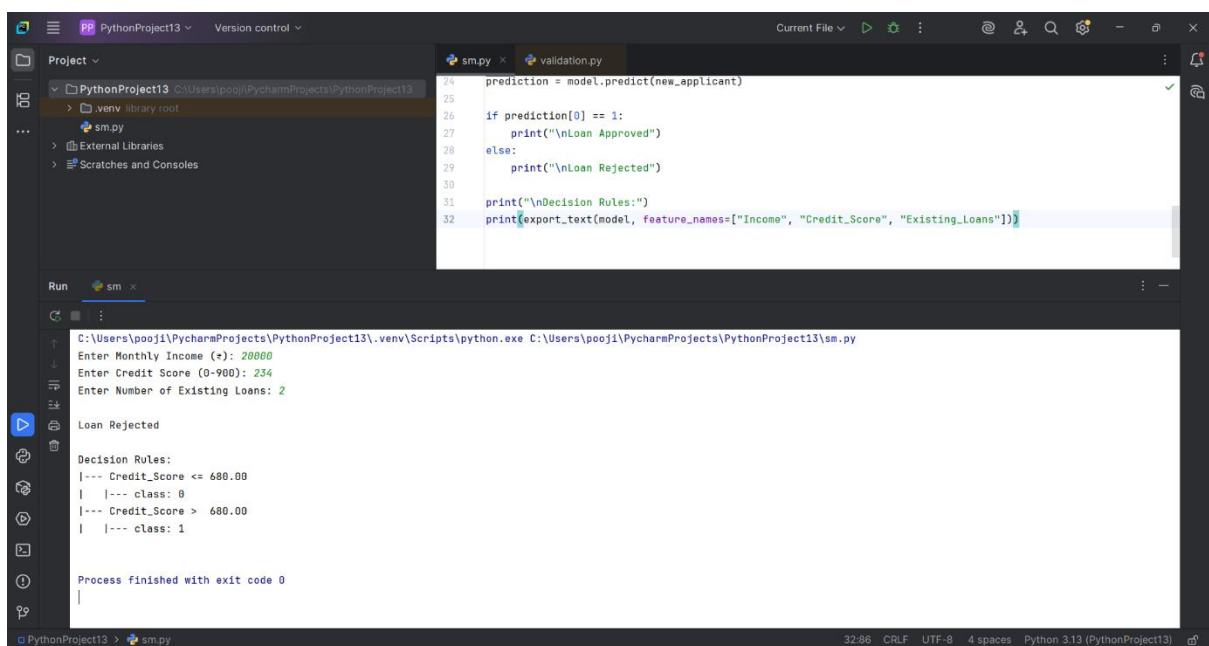
    print("\nLoan Rejected")

print("\nDecision Rules:")

print(export_text(model, feature_names=["Income", "Credit_Score", "Existing_Loans"]))

```

OUTPUT :



```

PythonProject13
├── PythonProject13
│   ├── venv
│   ├── sm.py
│   └── External Libraries
└── Scratches and Consoles

Run
sm
C:\Users\pooji\PycharmProjects\PythonProject13\venv\Scripts\python.exe C:\Users\pooji\PycharmProjects\PythonProject13\sm.py
Enter Monthly Income (+): 28000
Enter Credit Score (0-900): 234
Enter Number of Existing Loans: 2

Loan Rejected

Decision Rules:
|--- Credit_Score <= 680.00
| |--- class: 0
|--- Credit_Score > 680.00
| |--- class: 1

Process finished with exit code 0

```